

BTC•FUSION SYSTEM & MODEL ARCHITECTURE

How the pipeline is built, how a transaction is analysed end to end, what the models are, what they were trained on, and exactly where the machine learning sits.

CONTENTS

1	What the system is for	7	Training data and splits
2	System architecture	8	Attribution — network ⇔ chain
3	Data flow — the nine stages	9	Explainability
4	How one transaction is analysed	10	Measured results
5	Where the AI is	11	Limits and failure modes
6	Model specifications	12	The analyst interface

Every figure in this document is read from a build artefact, not quoted from memory. Unless stated otherwise the profile is **bulk** — 2,467,299 rows at a realistic 0.39% positive rate — and the source is `artifacts/bulk/manifest.json`, regenerated by `make reproduce`. Model `feature_version 7`, seed `20260826`, git `5a20a09`, backend `sklearn_histgb`, trained 2026-09-07.

1 • WHAT THE SYSTEM IS FOR

Commercial chain-analysis tools are a purchasable, solved product. None of them ingest packet capture, because a private company cannot lawfully tap an ISP. That capability gap is the reason this problem statement exists.

Every blockchain tool can tell you that money moved suspiciously. This one correlates the **network layer** — source IP, port, arrival timestamp — with the **chain layer** — TXID, addresses, amounts — to say *who moved it*, and refuses to answer when the evidence is shared infrastructure.

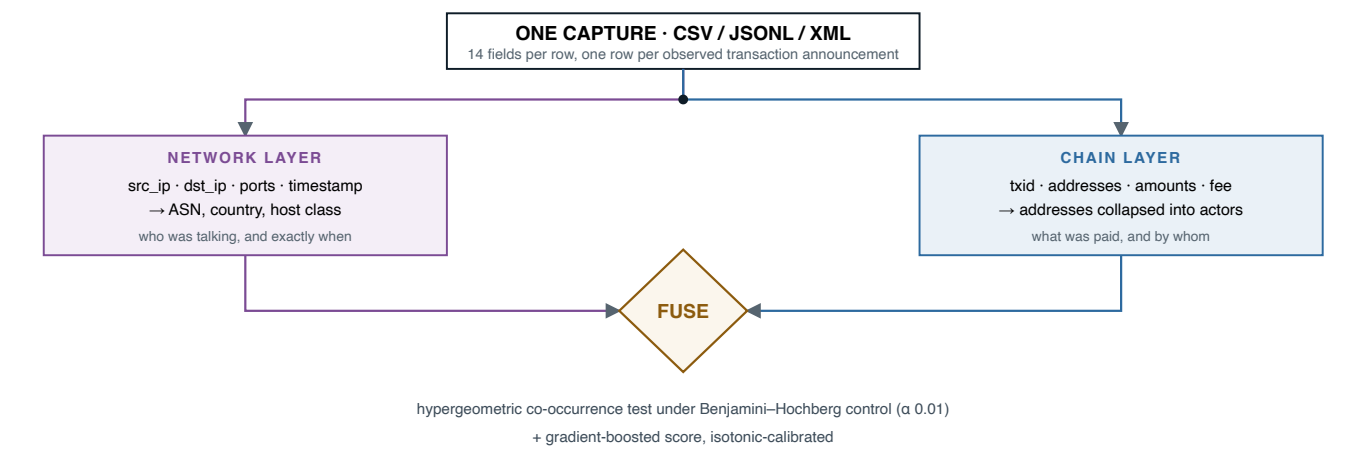


Figure 1. The thesis in one picture. Neither lane can answer the question alone; they meet once, at FUSE. Everything else in this document is how that happens.

The deployment context decides the architecture

The target is an air-gapped analyst workstation. That single constraint explains most of the engineering choices, and each one is a rejection of a more fashionable option:

CHOICE	REJECTED ALTERNATIVE	REASON
DuckDB, a file	PostgreSQL	A service that must be installed, started and kept alive on a machine with no network.
igraph, in memory	Neo4j	No analytical gain at this scale, and a second service to die mid-demo.
FastAPI in the same process as the models	A model-serving sidecar	No serialisation boundary, nothing to orchestrate.
Bundled DB-IP Lite ASN database	A GeoIP web API	Any lookup that leaves the machine breaks the entire premise.
Gradient-boosted trees	Graph neural network	Measured: see §6. SHAP is <i>exact</i> for trees.

The offline claim is enforced, not asserted. make offline-check greps the whole application package for https?:// and fails the build on a match. It cannot tell a citation from an endpoint, so URLs live in documentation and never in code. The container is additionally verified under --network none.

2 · SYSTEM ARCHITECTURE

One process. One container. Nine stages, executed in order, each writing something the next one reads and the provenance receipt can be checked against.

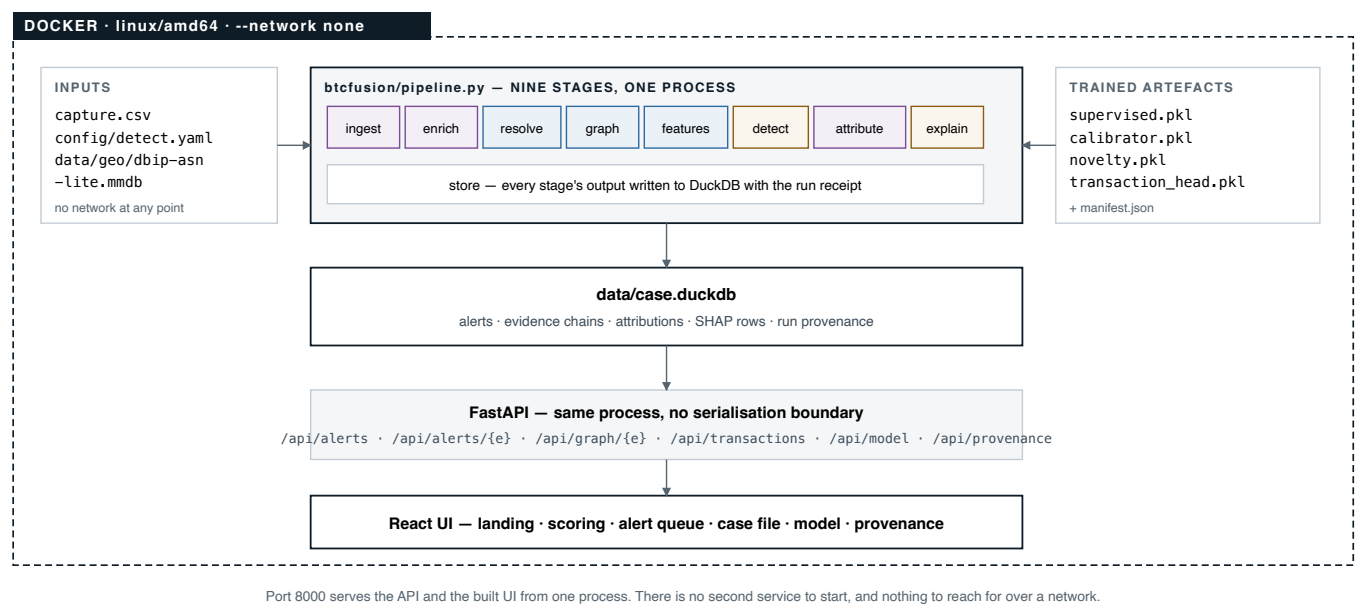


Figure 2. System architecture. Colour marks which layer a stage works in — **network**, **chain**, **model output** — the same palette the interface uses throughout.

Module map

Where each responsibility physically lives, so a reviewer can go from a claim in this document to the code that makes it true.

PACKAGE	RESPONSIBILITY
ingest/	<code>parsers.py</code> (CSV / JSONL / XML → one frame), <code>schema.py</code> , <code>mapper.py</code> , <code>enrich.py</code> (offline GeoIP / ASN)
graph/	<code>resolve.py</code> — common-input ownership; <code>build.py</code> — money graph and the entity × IP matrix
features/	<code>extract.py</code> — 77 engineered features, five of which correct for right-censoring; <code>embeddings.py</code> — the 64-d Node2Vec block
detect/	<code>supervised.py</code> (GBDT), <code>novelty.py</code> (IsolationForest + HDBSCAN), <code>typologies.py</code> (rules, evidence only), <code>transaction_head.py</code> , <code>fuse.py</code> (fusion + isotonic calibration), <code>pu.py</code> (Positive-Unlabelled learning, measured on Elliptic)
attribute/	<code>engine.py</code> , <code>diffusion.py</code> , <code>significance.py</code> , <code>infra.py</code> , <code>behaviour.py</code> , <code>confidence.py</code> — one module per step of §8
explain/	<code>narrative.py</code> — SHAP values into English; <code>report.py</code>
eval/	<code>metrics.py</code> (including the three queue orderings), <code>splits.py</code> , <code>labels.py</code> , <code>failures.py</code> , <code>sensitivity.py</code> , <code>external.py</code> (Elliptic), <code>elliptic_pp.py</code> , <code>diagnostics.py</code> (censoring versus drift)
generator/	The simulator: <code>actors.py</code> , <code>ledger.py</code> , <code>network.py</code> , <code>typologies.py</code> , <code>simulate.py</code> , <code>emit.py</code> , and <code>leak_test.py</code> — the gate of §7
store/ · api/	<code>dao.py</code> — DuckDB access; <code>api/main.py</code> — FastAPI, same process as the models

pipeline.py · train.py	The nine-stage run, and the training entry point that writes artifacts/<profile>/
ui/src/	App.tsx — shell and hash routing; api.ts — typed client; AlertQueue.tsx with QueueViews.tsx — the queue and its table / dossier / focus layouts; CaseFile.tsx, GraphView.tsx, ModelPanel.tsx, Provenance.tsx — §12
scripts/ · tests/	Offline wheel vendoring, GeolP fetch, container verification and the score fingerprint; 87 tests, including a cross-process determinism check

3 · DATA FLOW — THE NINE STAGES

What each stage consumes, what it computes, and what it leaves behind for the receipt. Timings are the measured bulk-profile run recorded in `manifest.json` → `timings`.

01	INGEST	raw file → one clean frame CSV, JSONL and XML all parse to the same internal frame, and all three are verified to produce identical clean-row counts. Rows failing validation are quarantined with a named reason and counted, never silently dropped — an analyst has to know what was excluded before trusting what was kept. ↳ <code>rows_read</code> · <code>rows_clean</code> · <code>quarantine reasons</code>	3.56 s
02	ENRICH	every IP → ASN, country, host class Lookups run against the DB-IP Lite database bundled inside the image. The host class (residential / mobile / hosting / CDN / VPN / Tor) is what later decides whether an attribution can be trusted or has to be suppressed. ↳ <code>receipt: mmdb:dbip-asn-lite.mmdb</code>	2.41 s
03	RESOLVE	1,810,005 addresses → 829,821 actors Common-input ownership: if one transaction spends from five addresses at once, one party held all five private keys. 635,510 co-spend edges plus 344,938 change edges, transitively closed. Mean cluster purity against ground truth is 0.9956. ↳ <code>mean 2.18 addr/actor</code> · <code>largest 953</code> · <code>688,853 singletons</code>	2.96 s
04	GRAPH	actor-to-actor flow, and entity × IP Two structures, not one: the money graph (PageRank, sampled betweenness) and, separately, the sparse entity × IP co-occurrence matrix the attribution test needs. Sparse rather than dense is what keeps 2.4M rows inside laptop memory. ↳ <code>295,330 graph nodes embedded</code>	0.90 s
05	FEATURES	141 numbers per actor Chain behaviour, timing rhythm, network posture, position in the money graph, plus 64 learned Node2Vec dimensions. Expressed as Polars expressions so the whole matrix computes in parallel rather than row by row. ↳ <code>feature_version 7</code> — the most expensive stage by far	86.46 s
06	DETECT	score every actor, then calibrate it Gradient-boosted trees score all actors; isotonic regression turns the raw score into a probability that means something, so 0.90 is roughly a nine-in-ten chance rather than merely 'higher than 0.80'. Ranking is the classifier's job alone. ↳ <code>fusion weights 1.00 / 0.00 / 0.00</code> · <code>threshold 0.75</code>	*
07	ATTRIBUTE	which IP is really theirs A hypergeometric test per actor-IP pair with Benjamini-Hochberg control at α 0.01, weighted by propagation-tree roots and penalised by infrastructure class. Where the evidence points at shared infrastructure it says nothing rather than guessing. ↳ <code>290,787 pairs tested</code> · <code>28,661 suppressed</code>	*
08	EXPLAIN	why this actor was flagged Exact SHAP — TreeExplainer, not an approximation — for the 60 alerts actually shown. There is no reason to explain 829,821 actors nobody will open. The top contributions become an English narrative an analyst can act on. ↳ <code>shap_top_k 8</code> · <code>explain_top_n 60</code>	*
09	STORE	write the case files and the receipt Alerts, evidence chains with real TXIDs, attributions, SHAP rows and the run provenance all go to DuckDB: the input SHA-256, the seed, the feature version, the model backend and the git commit. ↳ <code>every figure traces to an exact input and an exact model</code>	*

Table 1. The nine stages as executed, with the artefact each contributes to the provenance receipt. Timings are from the bulk training run; detect onward are reported per run rather than per stage.

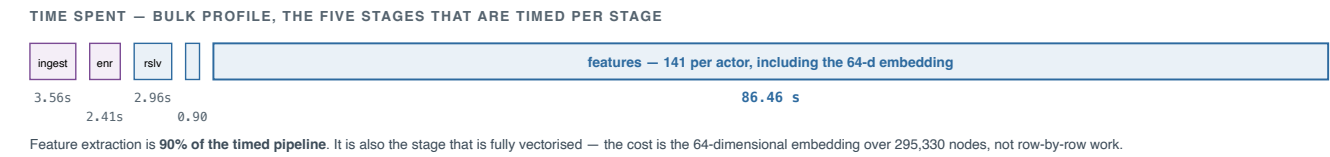


Figure 3. Where the time actually goes, to scale.

Degrade, never fail. If a stage raises, the run continues and the interface reports that component as unavailable. A stack trace in front of an analyst is the one unacceptable outcome, and a partial answer with an honest gap is more useful than no answer at all.

4 • HOW ONE TRANSACTION IS ANALYSED

The stages above describe the machinery. This is what actually happens to a single row of the capture, from the byte that arrives to the sentence an analyst reads.

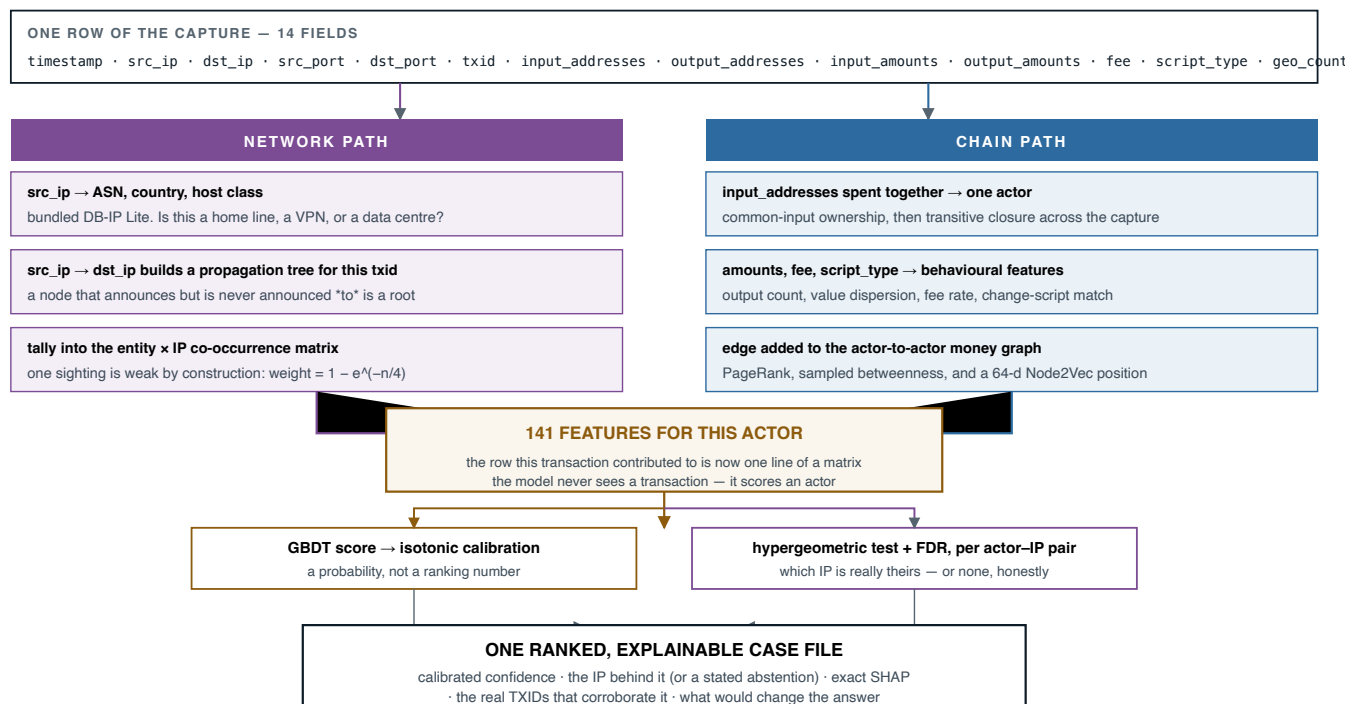


Figure 4. One row's journey. The critical structural point: **the model scores actors, never transactions** — a single transaction is evidence contributing to an actor's feature row. A separate transaction-level head exists for TXID ranking (§10).

Why not score transactions directly? A transaction has almost no behaviour — it has amounts and a timestamp. Laundering is a *pattern over time*: peel chains, rapid layering, structuring. Those are properties of an actor across many transactions, which is why the primary model works at the actor level and reaches PR-AUC 0.2998, while the transaction head reaches 0.0726 against a 0.0015 base rate. Both are reported; neither is hidden.

5 • WHERE THE AI IS

Five learned components, each doing a job that cannot be written as a rule. Every one is trained, versioned and recorded in the manifest — and the ablation in §10 proves that the lift comes from them and not from the rules beside them.

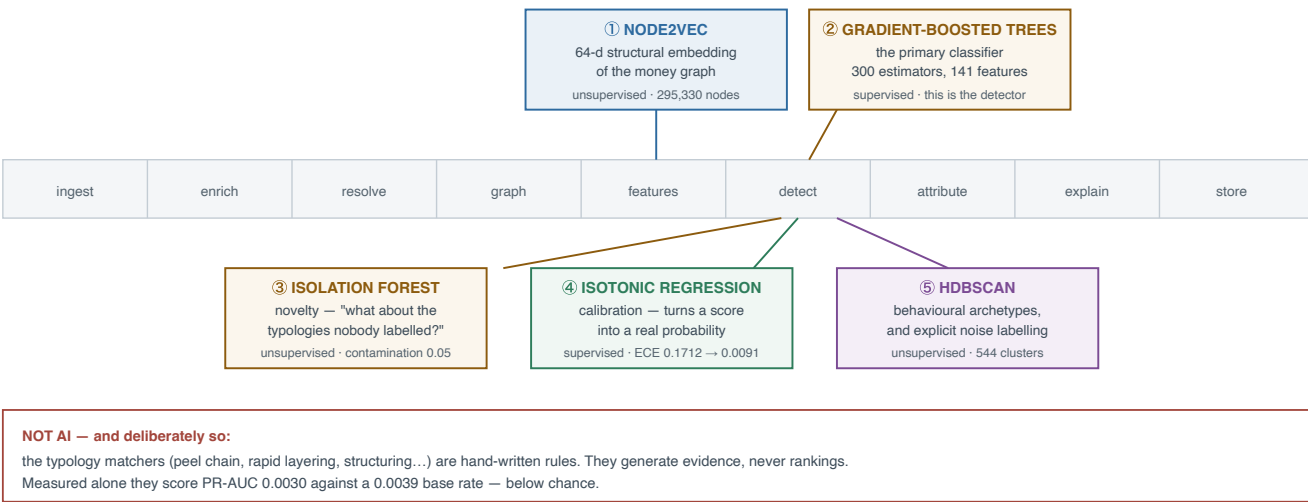


Figure 5. Every learned component, positioned at the stage it runs in. Three of the five are unsupervised, which matters for a domain where labels are scarce and adversaries change behaviour.

Answering the question directly

The AI is not a wrapper around a rules engine, and this is checkable rather than asserted. The ablation below is a row-by-row build-up of the shipped system, and each row is an entry in `metrics.json` → `ablation`:

WHAT IS SWITCHED ON	TEST PR-AUC	× BASE RATE	P@50	READING
Typology rules only	0.0030	0.77×	0.00	Worse than chance. Rules cannot rank.
+ unsupervised novelty	0.0080	2.1×	0.00	Signal appears, but not orderable.
+ GBDT, before censoring correction	0.2772	71×	0.92	The lift is here.
+ censoring-corrected features	0.2734	70×	0.92	A wash at this base rate; worth +0.042 on demo. Reported both ways.
+ Node2Vec embeddings	0.2998	77×	0.92	Learned topology adds 9.7% more.
+ isotonic calibration (display + gate)	0.2803	72×	0.86	Calibration costs 6.5% of ranking — which is why it no longer ranks.
= the queue the analyst sees (shipped)	0.2998	77×	0.92	Raw score orders it; the calibrated probability gates and displays it.

The base rate on this fold is 0.0039. Rules alone land *below* it. Everything above the base rate is contributed by the learned components — which is precisely what the problem statement's "a *working model*, not just rules" asks to be demonstrated.

A measurement that overturned our own design. The system originally fused three signals into one ranking score at 0.85 supervised / 0.05 novelty / 0.10 evidence. That survived the demo profile and collapsed at a realistic base rate: PR-AUC fell from 0.2803 to **0.1644** at only 5% novelty weight and to **0.0561** at 20% — 41% of the ranking lost for the first five points alone. The evidence term carries no ranking signal at 0.39% positives, and being near-binary it promoted a broad band of unremarkable actors above genuinely suspicious ones. Ranking is now the classifier's job alone — 1.00 / 0.00 / 0.00. Coverage of unlabelled typologies is bought differently: **8 of the 60 queue slots are reserved** for the highest-novelty actors the classifier did not surface. Blending corrupts every rank; reserving costs exactly the slots it uses. Full sweep in `metrics.json` → `fusion_sweep`.

6 · MODEL SPECIFICATIONS

PRIMARY CLASSIFIER

Family	Gradient-boosted decision trees
Reference backend	LightGBM
Shipped backend	sklearn_histgb
Estimators	300
Input features	141
Unit of prediction	actor (entity)
Training rows	177,126
Train duration	596.66 s
Feature version	7
Seed	20260826

The shipped artefacts record sklearn_histgb because they were trained on a macOS host without libomp. A fresh detector inside the Linux container resolves LightGBM — the same histogram algorithm. The manifest records which backend produced every number, so the two are never confused.

NODE2VEC EMBEDDING

Backend	ppmi_svd
Dimensions	64
Walks per node	8
Walk length	16
Return / in-out (p, q)	1, 1
Nodes embedded	295,330

NOVELTY & CLUSTERING

Novelty model	IsolationForest
Contamination	0.05
Archetypes	HDBSCAN
Min cluster size	25
Clusters found	544
Noise fraction	0.589

CALIBRATION & THRESHOLDS

Calibrator	Isotonic regression
Alert threshold	0.75
Max alerts	60
Novelty slots reserved	8
SHAP top-k	8

Why these models, and what was rejected

MODEL	JOB	WHY THIS ONE
Node2Vec (64-d)	Structural roles	Nobody labels "mixer-like topology". The embedding discovers it, and it runs CPU-only.
Gradient-boosted trees	Primary classifier	Best-in-class on tabular data, and SHAP is exact for trees rather than approximated.
IsolationForest	Novelty	Linear-time, no distance assumptions. Answers "what about patterns you never labelled?"
HDBSCAN	Behavioural archetypes	Does not need <i>k</i> , and explicitly labels noise — which is what an outlier is.
Isotonic regression	Calibration	Non-parametric, so it corrects arbitrary miscalibration shapes rather than an assumed one.

Graph neural networks were rejected on evidence, not taste. Weber et al. (2019) found tree ensembles outperformed a GCN on illicit-class F1 on exactly this task. GNNExplainer output is hard to turn into analyst-readable prose, and a GNN is likelier to memorise generator fingerprints — a real risk when the training data is synthetic. The structural signal a GNN would learn is captured instead by the Node2Vec features, whose contribution is measured separately in the ablation (+0.026 PR-AUC on bulk, +0.114 on demo). Closed-form message passing over the payment graph — the part of a GNN that carries signal, with no deep-learning dependency and exact SHAP kept — was also built and measured. It lowered PR-AUC, and was removed.

What the 141 features are

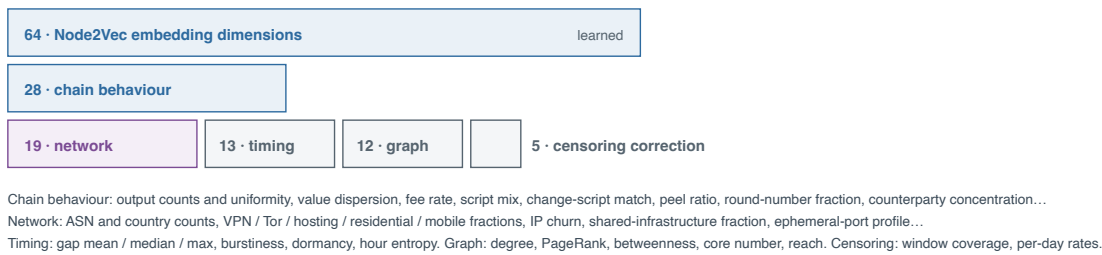


Figure 6. Feature composition, to scale. Nearly half the vector — 64 of 141 — is learned rather than engineered — the embedding is the part no analyst could have specified.

THE TEN FEATURES THE MODEL ACTUALLY LEANS ON

FEATURE	SHARE	WHAT IT CAPTURES
max_n_outputs	0.158	The widest fan-out this actor ever produced — the shape of a distribution step.
emb_2	0.075	Learned structural position. No hand-written meaning — that is the point.
degree_in	0.070	How many distinct counterparties pay this actor.
emb_4	0.067	As above.
mean_n_outputs	0.058	Whether fan-out is habitual or a one-off.
in_out_ratio	0.046	Collector versus distributor behaviour.
vpn_frac	0.035	Network layer. Share of announcements from VPN address space.
pagerank	0.026	How central this actor sits in the flow of money.
hosting_frac	0.026	Network layer. Share from data-centre ranges.
tor_frac	0.024	Network layer. Share from Tor exits.

Shares are normalised mean |SHAP| over a sample, because the shipped HistGradientBoosting backend exposes no split gain. Three of the top ten come from the network layer — the half of the data a blockchain explorer does not have. That is the correlation the problem statement asks for, appearing as measured feature importance rather than as a claim.

7 · TRAINING DATA AND SPLITS

No dataset was provided, and **no public dataset has a network layer**. So "collection" meant writing a defensible simulator — 1,701 lines, seeded, versioned — and then attacking it harder than a reviewer would.

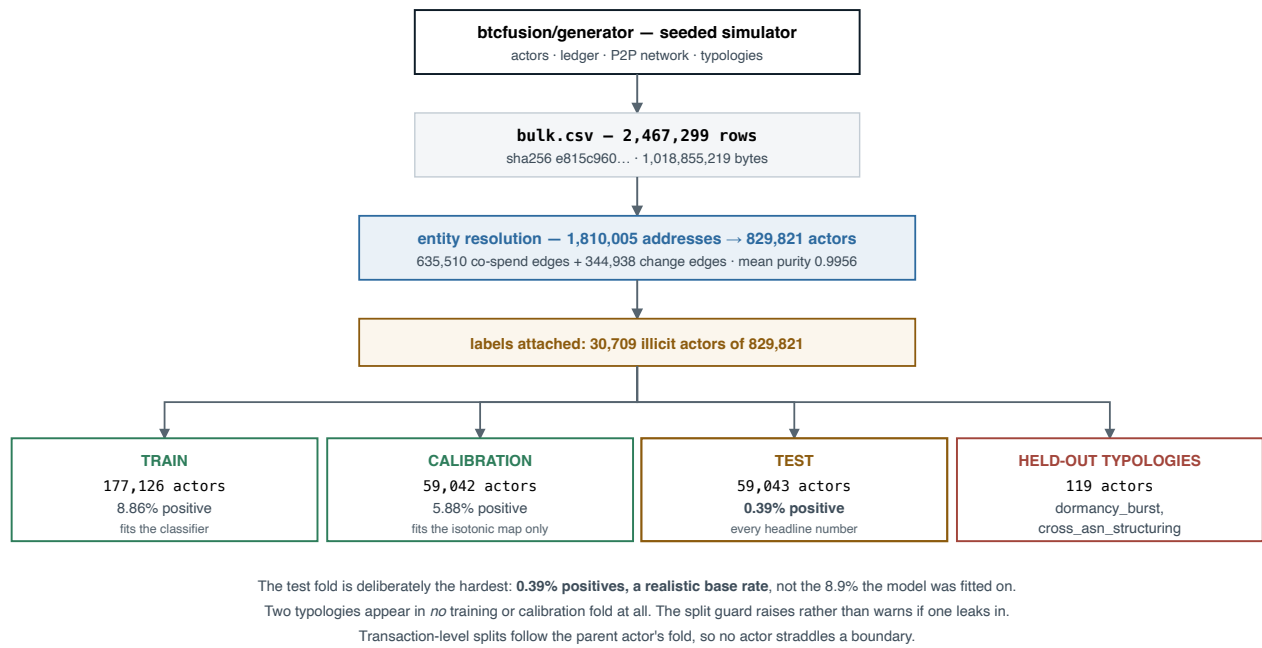


Figure 7. Data generation through to splits. Note the base-rate gradient across the folds — training at 8.86% and testing at 0.39% is deliberate, because the deployment reality is the latter.

The leak test — and the two defects it caught

We wrote the generator, so we know the answers, so every accuracy number is self-referential. A sharp reviewer attacks the data, not the model. So the build gates on a test that trains a classifier *only* on fields with no constructed path to the label, and requires it to score at the base rate.

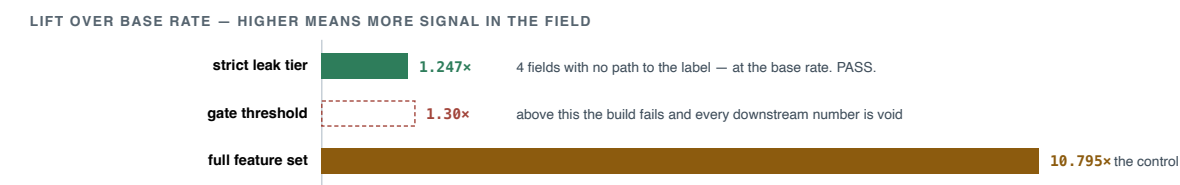


Figure 8. The leak test, from `artifacts/v1/leak_test.json`. The ~9× gap between the strict tier and the control is the evidence that detection comes from topology, timing and network structure rather than from an artefact of how the data was written.

It has earned its keep twice, and both catches are the kind only this test finds:

1. **Crime proceeds were drawn from a different amount distribution than licit payments** — a 1.7× shortcut. Fixed: proceeds now use the same payment-size distribution, with the archetype controlling count and dispersion, never absolute scale.
2. **Mule wallets drew their operating system from a different distribution than everyone else** — 1.2× on ephemeral-port features. The model was reading "no BSD hosts" as evidence of laundering.

Reproducible from the seed — and the defect that said otherwise

A published number is worth something only if someone else can regenerate it. The generator was documented as deterministic and was not: it chose an actor's announcing IP with Python's built-in string `hash()`, which is randomised per process. Two runs at the same seed agreed on every column except `src_ip` — the network half of the thesis. It now uses `zlib.crc32`, the figures in this document were regenerated after the fix, and `tests/test_determinism.py` checks the property *across a process boundary*, because inside a single process the bug is invisible.

Validated against real Bitcoin, not only our own data

Because the synthetic set is ours, the chain-side detector is additionally validated on real labelled Bitcoin. This proves nothing about the network layer — Elliptic has no IP data — and the document says so rather than blurring the two.

VALIDATION	DATA	RESULT	REFERENCE
Elliptic, transaction level	46,564 real labelled transactions, temporal split (train steps 1–34, test 35–49)	F1 0.7595 PR-AUC 0.8009	Weber et al. published 0.79
Elliptic++, actor level	96,023 real addresses, address-disjoint split, 5.56% base rate	PR-AUC 0.3745	the level we actually ship
Clustering heuristic itself	real Bitcoin co-spend clusters	99.79%	label agreement vs 82.09% shuffle control

Using the 77% of Elliptic everyone discards — measured, not adopted

Elliptic leaves 77% of its transactions unlabelled, and published work — including the baseline above — drops them. Positive-Unlabelled learning (Elkan & Noto, 2008) is the principled alternative: estimate how likely an illicit transaction was to be labelled at all, then re-weight the unknowns instead of discarding them. `detect/pu.py` implements it and `eval/external.py` compares it on the same temporal split.

ELLIPTIC TEST SPLIT	DROP THE UNKNOWNNS	POSITIVE-UNLABELLED
PR-AUC	0.8009	0.7710
ROC-AUC	0.9412	0.9493
Best F1	0.8259	0.7059
Unlabelled rows used in training	0	106,371

It loses wherever the top of the ranking matters. The estimate explains why: $c = 0.9157$, meaning roughly 92% of illicit transactions are *already* labelled, so the unknown pool is nearly all licit and there is little to recover. It is not used on the synthetic data at all — the generator knows every label, so $c = 1$ by construction. Keeping the simpler method is now a measurement rather than a convention.

8 · ATTRIBUTION — THE NETWORK ⇔ CHAIN CORRELATION

The component that answers the problem statement's load-bearing verb. Given an on-chain actor, which network identity is behind it — and how sure are we, honestly?

The problem it has to solve. Naive attribution says "whoever we saw announce it first is the originator". Koshy et al. (FC 2014) and Biryukov et al. (CCS 2014) showed P2P relay patterns leak origin — and Bitcoin Core responded. Modern diffusion applies an **independent randomised delay per peer**, so the first announcement any collector observes is frequently a relay, not the source. First-seen attribution has therefore been wrong since 2015. This engine models the defence instead of ignoring it.

①	entity × IP co-occurrence matrix How often each actor was seen announcing from each IP, counted across every announcement in the capture — not the first one.	engine.py
②	propagation-tree roots, per transaction src_ip → dst_ip edges with timestamps reconstruct who announced to whom. A node that announces but is never announced to within that transaction is a root — materially stronger evidence than “seen first”. Where no candidate survives, the transaction contributes no root evidence at all. Most designs ignore dst_ip entirely and cannot do this.	diffusion.py
③	hypergeometric significance vs a null model A busy relay co-occurs heavily with everyone. The real question is whether the association is stronger than chance given how much traffic that address carries and how many transactions that actor made . Benjamini–Hochberg FDR control is applied across the whole pair set at $\alpha 0.01$ — testing hundreds of thousands of pairs at an uncorrected $p < 0.01$ would hand back thousands of spurious links.	significance.py
④	infrastructure classification and penalty Two independent sources, because either alone is exploitable: the declared ASN type from the bundled database, and the observed number of distinct actors announcing from that address in this capture.	infra.py
⑤	behavioural timezone inference People keep hours. Which UTC offset makes this actor's activity histogram look most like a human waking, working and sleeping? Derived from behaviour, not from the GeoIP database — so when the two independent methods agree, that is real corroboration. A flat 24/7 histogram fits every offset equally and is reported as such.	behaviour.py
⑥	confidence, interval, counterfactuals The product of everything above. Where every candidate is shared infrastructure, the attribution is suppressed rather than reported at a confidence that would mislead. An attribution engine that always produces an answer is not an attribution engine.	confidence.py

Table 2. The attribution engine, six steps, one module each. The counterfactuals in step ⑥ are computed by re-running this same model with one input changed — so they are a property of the model, not authored text.

confidence =

statistical strength × repeat-observation weight × propagation-root bonus
× infrastructure penalty × timezone corroboration

Repeat-observation weight saturates as $1 - e^{-n/4}$: one sighting earns ~22% of full weight, eight earn ~86%. **One sighting can never on its own produce a confident attribution** — which is exactly the defence randomised diffusion provides.

INFRASTRUCTURE PENALTY

CLASS	MULTIPLIER
residential	1.00
mobile (CGNAT)	0.90
hosting	0.72
CDN	0.30
VPN	0.22
Tor exit	0.12

An ASN labelled residential that carries 400 distinct actors is a shared box whatever the database says — which is why the observed count is weighed alongside the declared class.

MEASURED, BULK PROFILE

Top-1 accuracy	0.9243
Random-choice baseline	0.3378
Top-3 / MRR	0.9251 / 0.9247
Attempt rate	98.94%
Abstentions	624
Mean candidates	3.641
Actor-IP pairs tested	290,787
Significant after FDR	280,911
Propagation-root pairs	80,569
Suppressed	28,661
FDR α	0.01

Abstentions are excluded from accuracy and reported separately. Suppressing an attribution that cannot be made honestly is correct behaviour, not a miss — an engine that always produces an answer is not an attribution engine.

The honest curve: accuracy depends on how much you can see

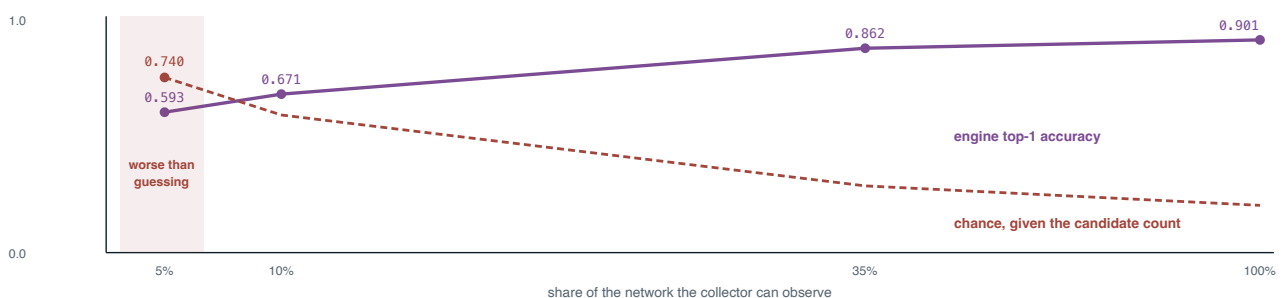


Figure 10. Below roughly 10% observation coverage the engine is worse than guessing — it is mostly seeing relays and picking among them confidently. This is reported rather than hidden: a single accuracy figure at an unstated coverage would be exactly the overclaim this module exists to avoid.

9 • EXPLAINABILITY

Four layers, because "explainable" is a graded deliverable and not garnish. Each answers a different question an analyst actually asks.

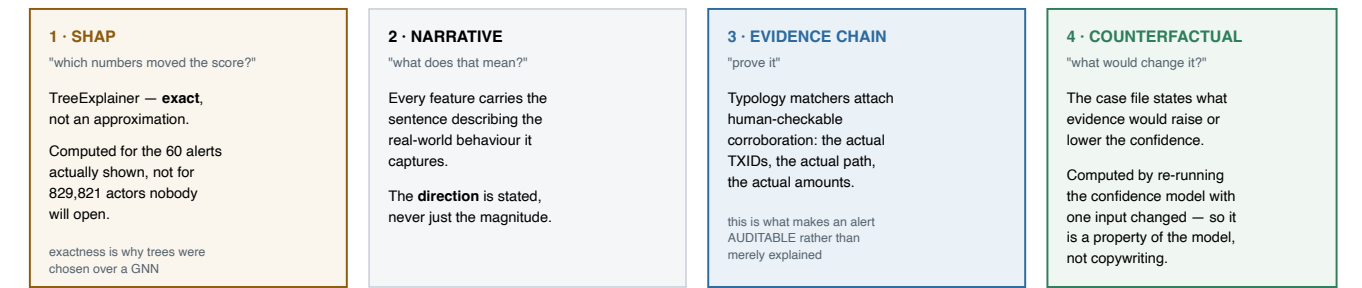


Figure 11. The four explainability layers. Layers 1 and 4 are computed from the model; layer 3 is checkable against the chain itself.

Stated openly: explanation faithfulness is not measured. SHAP values are shown to analysts, but no deletion or insertion test proves the highlighted features actually drive the prediction. This is the one item from the original plan that was never built, and it is recorded as an open item rather than quietly omitted.

10 · MEASURED RESULTS

Everything below is read from `artifacts/bulk/manifest.json`, regenerated by `make reproduce` from the seed. Where a metric can mislead at this base rate, its baseline is printed beside it.

Entity-level detection — the primary model

FOLD	N	POSITIVE RATE	PR-AUC	BASELINE	ROC-AUC	ECE	MCC
train	177,126	0.0886	0.9852	0.0886	0.9977	0.0230	0.8427
calibration	59,042	0.0588	0.8470	0.0588	0.9565	0.0075	0.7862
test	59,043	0.0039	0.2803	0.0039	0.7072	0.0381	0.3417

Read the test row, not the train row. The PR-AUC above is the calibrated probability's, 0.2803; the ordering the analyst is actually shown scores **0.2998** against a 0.0039 base rate, **77× chance**. On that shipped ordering precision@10 is 1.00, P@25 is 0.96 and P@50 is 0.92 — which is what an analyst working a 60-item queue actually experiences.

Accuracy is printed beside its own baseline on purpose. Test accuracy is **0.9953**. An all-negative classifier that flags nothing at all scores **0.9961** — better. That is exactly why accuracy never judges this system, and why MCC and PR-AUC do.

What the ranking figures measure

Every ranking figure here scores **the order an analyst is actually shown**, which is not the same as scoring the calibrated probability. Isotonic calibration is a step function that clusters near its 0.97 ceiling, so the queue ranks on the raw fused score at full resolution, uses value moved only to break an exact tie, and lets the calibrated probability do the two jobs it is good at: gating the queue at 0.75, and being displayed.

ORDERING OF THE TEST FOLD	BULK PR-AUC	P@10 · P@25 · P@50	DEMO PR-AUC	P@10 · P@25 · P@50
Raw score, value breaks exact ties — shipped	0.2998	1.00 · 0.96 · 0.92	0.5889	1.00 · 1.00 · 0.98
Calibrated probability	0.2803	0.60 · 0.84 · 0.86	0.5761	0.80 · 0.88 · 0.86
Calibrated, banded to two decimals, value inside a band — previous	0.2720	0.70 · 0.84 · 0.86	0.5606	0.90 · 0.64 · 0.82

The banded ordering was the original design, on the reasoning that two leads at 0.93 are indistinguishable so the bigger money is the better lead. Measured, it is the worst of the three on both profiles: on bulk it gives up thirty points of precision in the first ten leads and twelve in the first twenty-five. All three orderings are written to `metrics.json` → `results.*.queue` on every training run, so the choice stays evidenced.

Why the late fold is harder — censoring, not drift

Discrimination falls across the temporally forward splits: bulk ROC-AUC is 0.9977 on train, 0.9565 on calibration and 0.7072 on test. The base rate falls with it, but ROC-AUC does not depend on the base rate, so something about the late fold is genuinely harder. Two explanations predict different shapes. **Drift** — the model ageing — predicts a flat deficit. **Right-censoring** predicts recovery as observation time grows: a capture ends, so an actor first seen a week before the end has only a week of behaviour to describe it, and a

temporally forward split puts exactly those actors in the test fold. `eval/diagnostics.py` stratifies the fold to tell them apart.

TIME BETWEEN FIRST ACTIVITY AND END OF CAPTURE	DEMO DAYS	ROC-AUC	BULK DAYS	ROC-AUC
Shortest quartile	0.0 – 7.4	0.761	2.7 – 12.5	0.559
Second quartile	7.4 – 13.3	0.865	12.5 – 19.8	0.664
Third quartile	13.3 – 18.5	0.911	19.8 – 26.3	0.727
Longest quartile	18.5 – 23.8	0.938	26.3 – 32.3	0.767

Monotone in the observation window on both profiles: it is censoring. On bulk the automated verdict in `metrics.json` reads *indeterminate*, because its primary check — actors still active in the final 24 hours — has only two members there; the quartile stratification is the evidence, and it replicates. Actors that never sent a transaction score at chance (ROC-AUC 0.490 on demo, 0.497 on bulk), because almost every feature is sender-side: a low score on such an actor means *unranked*, never cleared.

Five features normalise each aggregate by the window that was actually available. They are worth +0.042 PR-AUC on demo (0.4323 → 0.4746) and −0.004 on bulk (0.2772 → 0.2734) — a gain on one profile and a wash on the other, reported both ways. The raw window length is deliberately **not** a feature: the generator front-loads campaigns, so calendar position would teach the model that late means licit. The leak test passes with the correction in place.

Calibration — does 0.90 mean 90%?

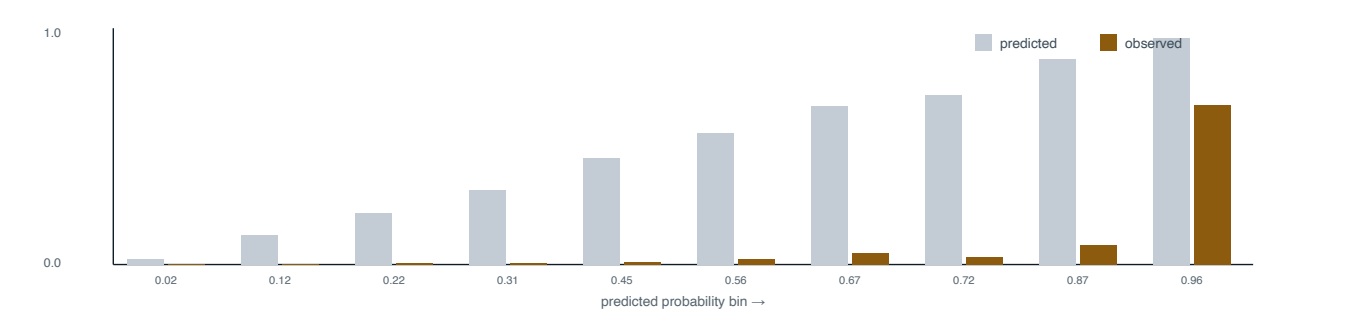


Figure 12. Reliability on the test fold, from `metrics.json` → `reliability_curve`. The top bin predicts 0.9586 and observes 0.6735 across 98 actors. Isotonic regression cut calibration error on the calibration fold from **0.0984 to 0.0075**; on the far harsher test fold it lands at 0.0381, down from 0.1972 uncalibrated. The model is still optimistic in the middle bins — the test fold's base rate is fifteen times lower than the fold the calibrator learned on — and the chart shows that rather than hiding it.

Transaction-level head

METRIC	VALUE	NOTE
Transactions scored	595,929	2,596 illicit; splits follow the parent actor's fold
Test set	125,308	192 positives — a 0.0015 base rate
PR-AUC	0.0726	vs 0.0015 baseline — 48x chance
ROC-AUC	0.8317	
Precision@10	0.90	the top of the ranked TXID list is usable

Weaker than the actor-level model, and necessarily so — a transaction has amounts and a timestamp, while laundering is a pattern across many transactions. Both are shipped because the problem statement asks for flagged *wallets and transactions*.

Entity resolution and clustering

Addresses	1,810,005	Singleton actors	688,853
Actors recovered	829,821	Mean cluster purity	0.9956
Co-spend edges	635,510	Impure clusters	4,267
Change edges	344,938	Behavioural archetypes	544
Mean addresses / actor	2.18	Noise fraction	0.5387
Largest actor	953 addresses	Illicit actors labelled	30,709

Two levels of clustering, as the problem statement asks: **identity** clustering (addresses → actors, by common-input ownership) and **behavioural** clustering (actors → 544 archetypes, by HDBSCAN). They answer different questions and are never conflated.

11 • LIMITS AND FAILURE MODES

A system that cannot say where it breaks has not been evaluated. These are read from the artefacts, not softened.

Where the model actually fails

ACTOR	KIND	SCORE	WHY
ENT-254513	false positive	0.97	A licit short-lived pass-through wallet whose profile sits in the overlap between transient behaviour and laundering topology.
ENT-432299	false positive	0.97	Announced almost entirely from shared infrastructure, which lifts network-layer features without the chain layer supporting the flag.
ENT-324757	false positive	0.97	Another transient wallet in the same overlap region.
ENT-31606	false negative	0.01	Genuinely rapid_layering, but too few transactions to establish the topology the model needs.
ENT-325798	false negative	0.01	Genuinely rapid_layering; same cause.
ENT-25041	false negative	0.01	A genuine extortion actor outside every named typology, with too little history to show a topology.

The pattern is consistent and worth stating plainly: **the model needs history**. An actor with a handful of transactions cannot show a topology, so a real criminal who has just started is exactly the case this system is weakest on.

Known limits

LIMIT	WHAT IT MEANS IN PRACTICE
Attribution collapses below ~10% observation coverage	Worse than guessing — see Figure 10. Coverage must be stated with any accuracy claim.
Training data is synthetic	Mitigated by the leak test and by two real-data validations, but the network layer itself has no real-world validation set in existence.
Explanation faithfulness unmeasured	SHAP is shown; no deletion/insertion test proves it drives the prediction.
Scaling is superlinear	4.4× the data cost 9.5× the time in the features stage — 9.10 s on demo, 86.46 s on bulk — which dominates the run.
Calibration is optimistic in mid-range bins	Visible in Figure 12. Trust the ordering more than the absolute number between 0.3 and 0.7.
Transaction head is weak in absolute terms	PR-AUC 0.0726. Usable at the top of the list (P@10 0.90), not as a filter.

What makes every number in this document checkable

Each run writes a provenance receipt: the input file's SHA-256, the seed, the feature version, the model backend and the git commit. The trained artefacts carry the same. The container build additionally re-derives the host's score fingerprint inside Linux and **fails the build if it drifts** — because the container installs scikit-learn 1.7.2 while the shipped artefacts were pickled by 1.9.0, and "the scores are bit-identical" is a fact about two specific versions rather than a property of the design.

Input

bulk.csv • sha256 e815c960431d1682...

Size	1,018,855,219 bytes
Seed	20260826
Feature version	7
Git commit	5a20a09
Backend	sklearn_histgb
Trained	2026-09-07T11:10:58Z

Reproducing this document's figures

COMMAND	WHAT IT REGENERATES
make reproduce	Every number in §5, §6, §7, §8 and §10 from the seed.
make verify-all	The machine-checkable subset of the compliance matrix in one pass.
make leak-test	Figure 8 — and gates the build at 1.30x.
make validate-external	Elliptic, 46,564 real transactions.
make validate-elliptic-pp	Elliptic++, 96,023 real addresses.
make docker-build && make docker-verify	Five offline steps under --network none on linux/amd64.
make offline-check	Fails on any URL in application code.

12 · THE ANALYST INTERFACE

The interface is what an analyst actually uses: one offline process serving both the API and the built React application, opening in the light theme. It is organised around a single question — which of these leads do I open first, and why is each one here?

SCREEN	ROUTE	WHAT IT IS FOR
Landing	/	The thesis, a drop zone for a capture, and three bundled samples (judge, demo, bulk). The live diffusion field behind it draws the problem the system solves: a peer announces a transaction and the rest relay it after randomised delays.
Scoring	while a run is active	The nine stages as a track, each explained in plain and technical language while it runs.
Alert queue	#/alerts	The ranked leads — described below.
Case file	#/case/{entity}	The investigative document for one lead: assessment, calibrated confidence, what would change it, SHAP attributions, network attribution, link analysis, activity timeline, evidence chain with real TXIDs, HTML export, confirm and dismiss.
Model	#/model	Scorecard, ablation, reliability diagram, held-out typologies, leak test, real-data validation, attribution against coverage, resolution quality, failure gallery, feature importance, split integrity.
Provenance	#/provenance	The run manifest — input SHA-256, seed, feature version, backend, git commit — plus ingest integrity and per-stage timings.

The alert queue

The queue opens on the reduction the system exists to produce — rows ingested, actors resolved, ranked leads — with the ingest receipt one click away rather than occupying the screen. Every lead then carries three **evidence marks**: **chain** when a named typology corroborates it, **network** when the attribution engine named an IP, and **model** for the classifier's own call or a reserved novelty slot. On a demo-profile run in data/case.duckdb, 41 of the 60 leads carry a typology, 3 carry a resolved network identity and 8 occupy novelty slots.



2,467,299 → 829,821 → 60

rows ingested actors resolved ranked leads

EVIDENCE MARKS – CHAIN · NETWORK · MODEL

- | | |
|---|---|
|  | a named typology corroborates the flag; no owner was named |
|  | typology and an IP the engine was willing to name — rare by design |
|  | the model's own call, or a reserved novelty slot |

Read down the column, not across a row. The middle mark lights up on a handful of leads in sixty, which makes the engine's refusal to guess visible instead of implied.

THREE LAYOUTS OF THE SAME SIXTY LEADS



table — compare a value down a column



dossier — read the written cases in order



focus — one lead, at presentation size

Figure 13. The alert queue. The funnel states the whole system in one line; the evidence marks say what stands behind each lead before it is opened; the three layouts trade comparison for reading.

LAYOUT	OPTIMISED FOR	WHAT EACH LEAD SHOWS
table	Comparing a value down a column across all sixty leads. The default.	Rank, evidence marks, entity with address · transaction · IP counts, pattern, attribution, confidence with its interval, novelty, and value moved with a log-scaled magnitude bar. Numeric columns sort.
dossier	Reading the queue in order.	Rank in the margin, entity as a heading, the written narrative with its announcing-host sentence set apart in the network colour — the part that differs from lead to lead — then figures and attribution.
focus	One lead at a time, with someone watching.	The lead at presentation size; a strip of all sixty leads coloured by evidence, clickable; narrative; figures; the five SHAP drivers; attribution; evidence chain with TXIDs; the link-analysis graph beside it; confirm and dismiss.

Controls: / focuses search, j / k or the arrow keys move, Enter opens the case file. Filters cover typology, host class and unreviewed leads; a confidence-threshold slider states how many leads it shows and how many it suppresses; an entities / transactions switch exposes the transaction-level head. Confirm and dismiss post to /api/alerts/{entity}/verdict and adjust only the next run's presentation order — dismissed -0.25, confirmed +0.10 on the ranking score. They never retrain the model, so a triage tool cannot quietly learn one analyst's habits.

Link analysis

The server extracts a k-hop subgraph with a hard 1,500-node cap and says when it has capped. Positions are computed directly rather than by a force-directed layout, so a case draws the same way every time, and every visual channel carries data.

CHANNEL	ENCODES
Ring	Hop distance from the subject, which sits at the centre.
Side	Left: counterparties the subject received value from. Right: counterparties it sent value to. A branch keeps the side of the first hop that created it.
Position on an arc	Value rank — the largest flows sit on the horizontal axis.
Node size and hue	Alert score, stretched over the run's observed range so the colour means the same thing on every case file.
Edge width	Log of value moved, because values span several orders of magnitude.
Marching dashes	The traced path, moving in the direction the money moved. Off under reduced motion.
Time scrubber	Replays edges in the order they first carried value, labelled with the real timestamp. Edge timestamps are stored in entity_edges; a run recorded before they were gets no scrubber rather than a broken one.

Hovering a node isolates its neighbourhood; clicking an actor traces its path back to the subject and offers its case file; the hop control switches between one, two and three hops; and *wallets* adds the subject's own addresses and transactions as membership links.

API

ENDPOINT	RETURNS
GET /api/health	Liveness, whether trained artefacts are present, backend and threshold.
GET /api/runs · /api/runs/latest	Run history and the latest run's ingest receipt.
POST /api/runs · GET /api/jobs/{id}	Start scoring a capture, and follow its progress.
POST /api/upload · GET /api/samples	Score an uploaded capture; list the bundled ones.
GET /api/alerts	The queue, with optional typology, country, host-class and review filters.

GET /api/alerts/{entity}	The case file: alert, evidence, attribution candidates, SHAP rows, behaviour profile, transactions.
POST /api/alerts/{entity}/verdict	Confirm or dismiss.
GET /api/alerts/{entity}/export	A self-contained HTML case file with the provenance embedded.
GET /api/graph/{entity} · .../expand	The capped k-hop subgraph with scores and edge timestamps; the entity's own wallets and transactions.
GET /api/transactions	Transaction-level alerts.
GET /api/model · /api/provenance · /api/quarantine	Model transparency, the run manifest, and rows rejected at ingest.

Built with

React and TypeScript, built by Vite into `ui/dist/`, which is committed so a fresh clone serves the interface without a Node toolchain. Tailwind maps to design tokens defined once in `index.css`; Cytoscape draws the graph; the small charts are hand-written SVG rather than a charting library; fonts are bundled locally, because nothing on an air-gapped machine can be fetched.

BTC·Fusion — SIH 2026 · PS 26146 · NTRO · Blockchain & Cybersecurity. Deeper treatments live beside the code: `docs/technical_writeup.md` (approach, model choice, evaluation), `docs/attribution_method.md` (the statistics), `docs/generator_parameters.md` (every synthetic-data parameter and its justification), `docs/model_card.md` (intended use and failure modes), `docs/external_validation.md` (what each validation does *not* prove).